EXPERIMENT -4

AIM- Using the Employee schema, write queries with INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. Compare execution plans using EXPLAIN.

OBJECTIVES

a. To understand and implement different types of SQL joins.
b. To retrieve related data using INNER JOIN and LEFT JOIN.
c. To understand and implement a self-join on the Employee table.
d. To perform a 3-way join between Employee, Department, and Project tables.
e. To understand and implement correlated subqueries.
f. To use the EXISTS operator for checking the existence of related records.
g. To simulate the functionality of INTERSECT and EXCEPT using SQL queries.
h. To analyze and compare the execution plans of SQL queries using EXPLAIN.

THEORY

INNER JOIN

An INNER JOIN returns only those records that have matching values in both tables based on the specified join condition. It is used to combine related information from two or more tables.

For example, the Employee and Department tables can be joined using the common DeptID attribute to display employees along with their departments.

LEFT JOIN

A LEFT JOIN returns all records from the left table and the matching records from the right table. If no matching record exists in the right table, NULL values are returned for the columns of the right table.

It is useful when all records from the primary or left-side table must be retained, even if they do not have corresponding records in the other table.

SELF-JOIN

A self-join is a join in which a table is joined with itself. Different aliases are assigned to the same table so that its records can be compared with one another.

It can be used to find relationships between employees, such as employees working in the same department or employees having similar salaries.

3-WAY JOIN

A 3-way join combines three tables in a single query using related attributes.

In the Employee schema, the Employee table can be joined with both Department and Project tables. This allows information such as employee name, department name, and project name to be retrieved together.

CORRELATED SUBQUERY

A correlated subquery is a subquery that depends on a value from the outer query. Unlike an independent subquery, it is logically evaluated for each row considered by the outer query.

Correlated subqueries are useful when the result of the inner query needs to be compared with the current row of the outer query, such as finding employees whose salary is greater than the average salary of their own department.

EXISTS

The EXISTS operator checks whether a subquery returns at least one row.

It returns TRUE when the subquery produces one or more matching records and FALSE when no matching record exists. It is commonly used to determine whether a related record exists in another table.

SIMULATED INTERSECT-

INTERSECT returns records that are common to the results of two queries.

When INTERSECT is not supported by a particular SQL system, its functionality can be simulated using alternatives such as INNER JOIN or EXISTS. The columns and data types involved in the comparison should be compatible.

SIMULATED EXCEPT-

EXCEPT returns records that are present in the first query result but absent from the second query result.

When EXCEPT is not supported, it can be simulated using NOT EXISTS or an appropriate LEFT JOIN with a NULL check

EXPLAIN

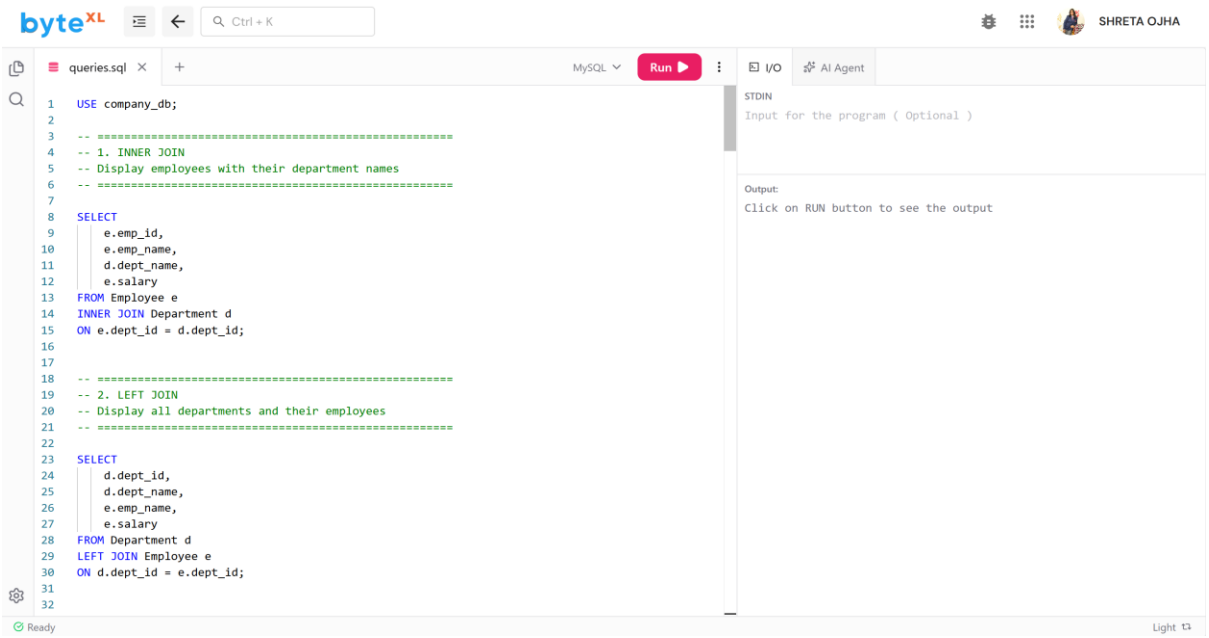
EXPLAIN is used to display the execution plan that the database optimizer intends to use for a SQL query.

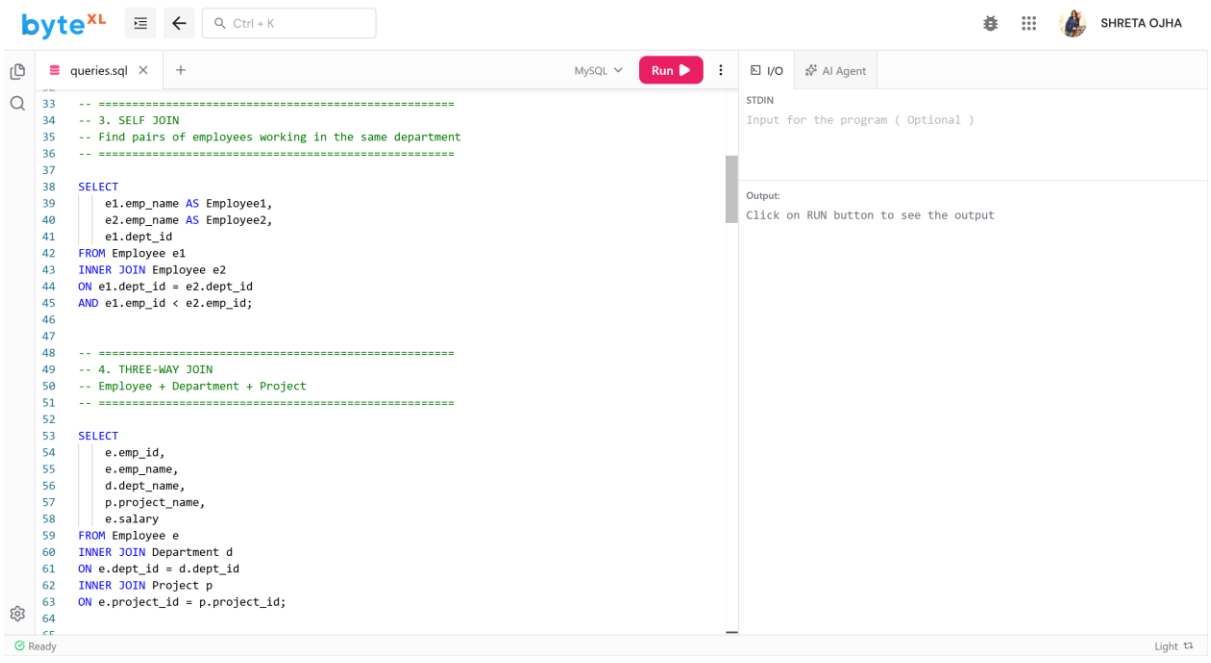
It provides information about how tables will be accessed and joined, such as:

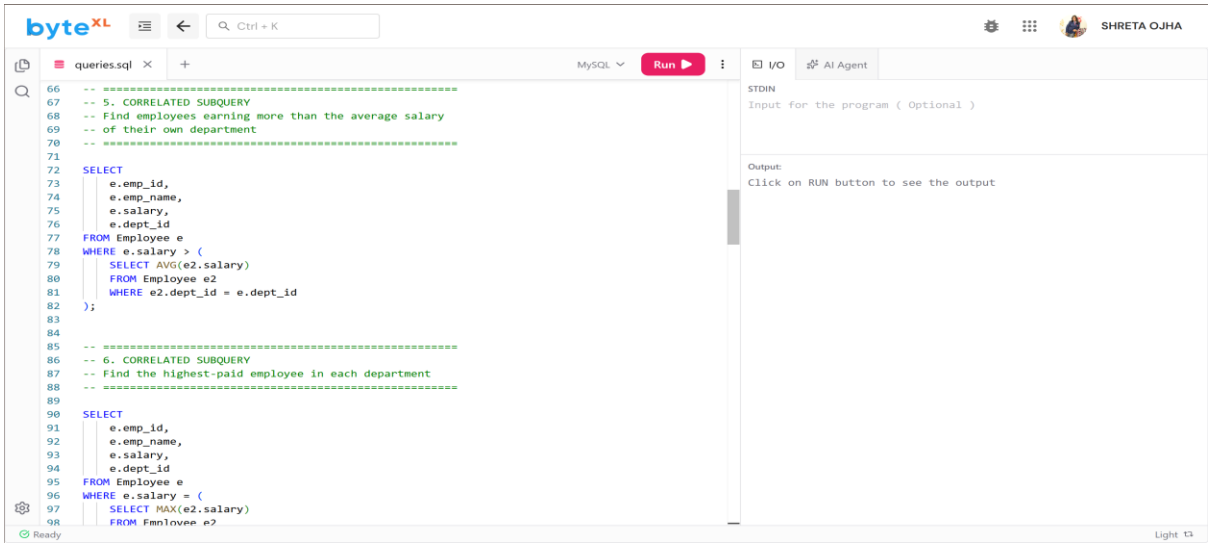
- Table scans
- Index usage
- Join methods
- Estimated number of rows
- Filtering operations
- Query execution cost

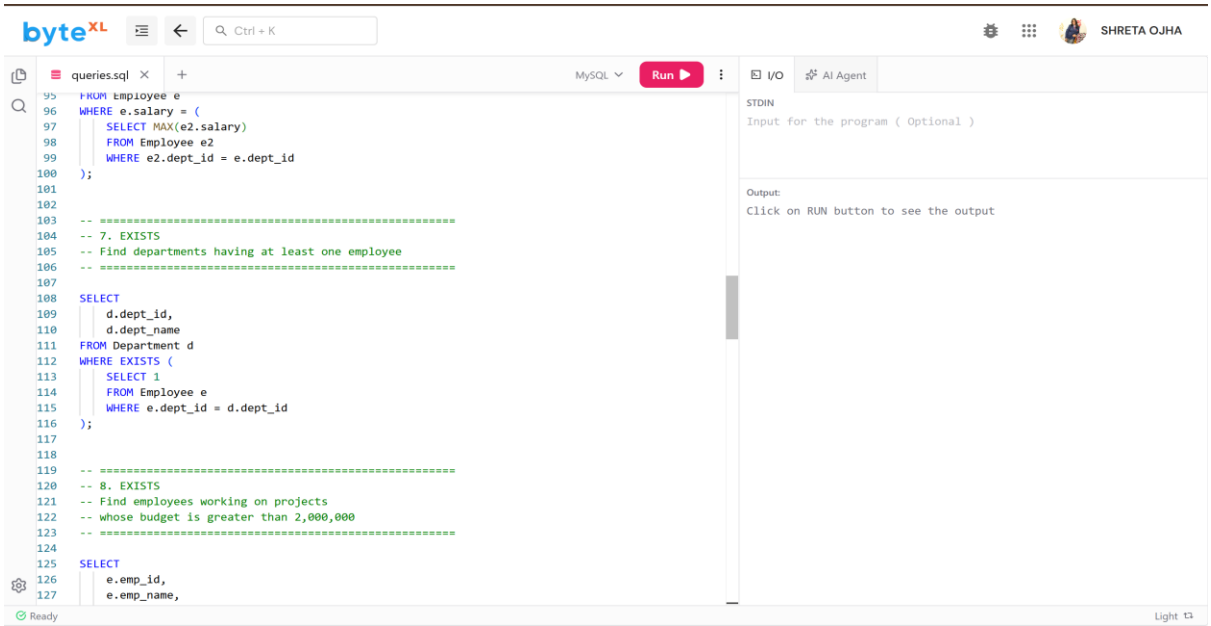
Comparing execution plans helps in understanding query performance and identifying inefficient queries. An execution plan with fewer scanned rows or an appropriate index access path may indicate a more efficient execution strategy, although the exact interpretation depends on the database system and data distribution.

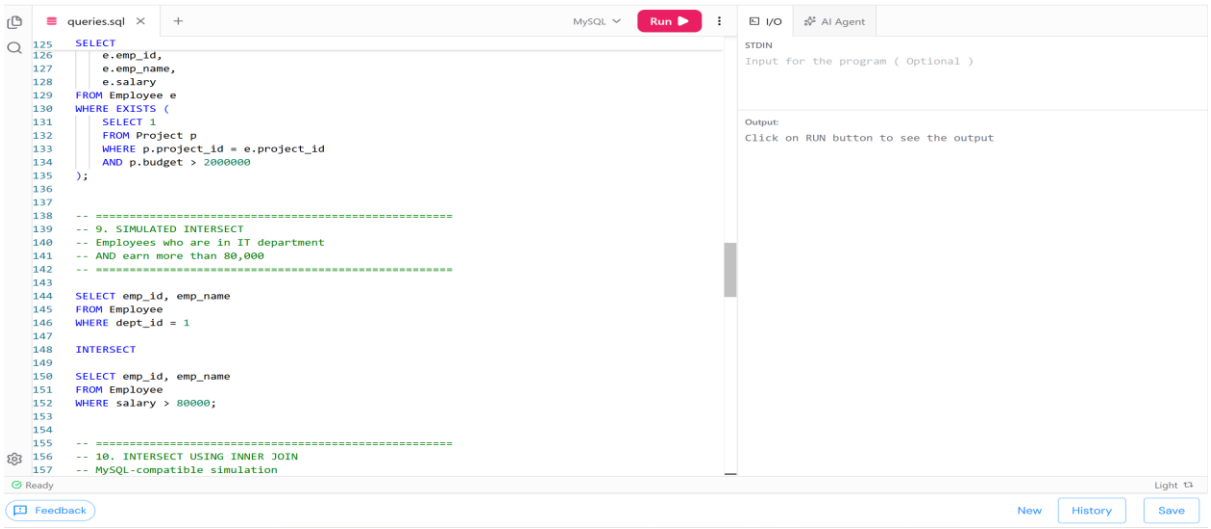
SOURCE CODE




byteXL
Ctrl + K
SHRETA OJHA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
Ready
Light
-- 3. SELF JOIN
-- Find pairs of employees working in the same department
SELECT
e1.emp_name AS Employee1,
e2.emp_name AS Employee2,
e1.dept_id
FROM Employee e1
INNER JOIN Employee e2
ON e1.dept_id = e2.dept_id
AND e1.emp_id < e2.emp_id;
-- 4. THREE-WAY JOIN
-- Employee + Department + Project
SELECT
e.emp_id,
e.emp_name,
d.dept_name,
p.project_name,
e.salary
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id
INNER JOIN Project p
ON e.project_id = p.project_id;


byteXL
Ctrl + K
SHRETA OJHA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
Ready
Light
-- 5. CORRELATED SUBQUERY
-- Find employees earning more than the average salary
-- of their own department
SELECT
e.emp_id,
e.emp_name,
e.salary,
e.dept_id
FROM Employee e
WHERE e.salary > (
SELECT AVG(e2.salary)
FROM Employee e2
WHERE e2.dept_id = e.dept_id
);
-- 6. CORRELATED SUBQUERY
-- Find the highest-paid employee in each department
SELECT
e.emp_id,
e.emp_name,
e.salary,
e.dept_id
FROM Employee e
WHERE e.salary = (
SELECT MAX(e2.salary)


byteXL
Ctrl + K
SHRETA OJHA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
Ready
Light
FROM Employee e
WHERE e.salary = (
SELECT MAX(e2.salary)
FROM Employee e2
WHERE e2.dept_id = e.dept_id
);
-- 7. EXISTS
-- Find departments having at least one employee
SELECT
d.dept_id,
d.dept_name
FROM Department d
WHERE EXISTS (
SELECT 1
FROM Employee e
WHERE e.dept_id = d.dept_id
);
-- 8. EXISTS
-- Find employees working on projects
-- whose budget is greater than 2,000,000
SELECT
e.emp_id,
e.emp_name,




```sql
SELECT
    e.emp_id,
    e.emp_name,
    e.salary
FROM Employee e
WHERE EXISTS (
    SELECT 1
    FROM Project p
    WHERE p.project_id = e.project_id
    AND p.budget > 2000000
);

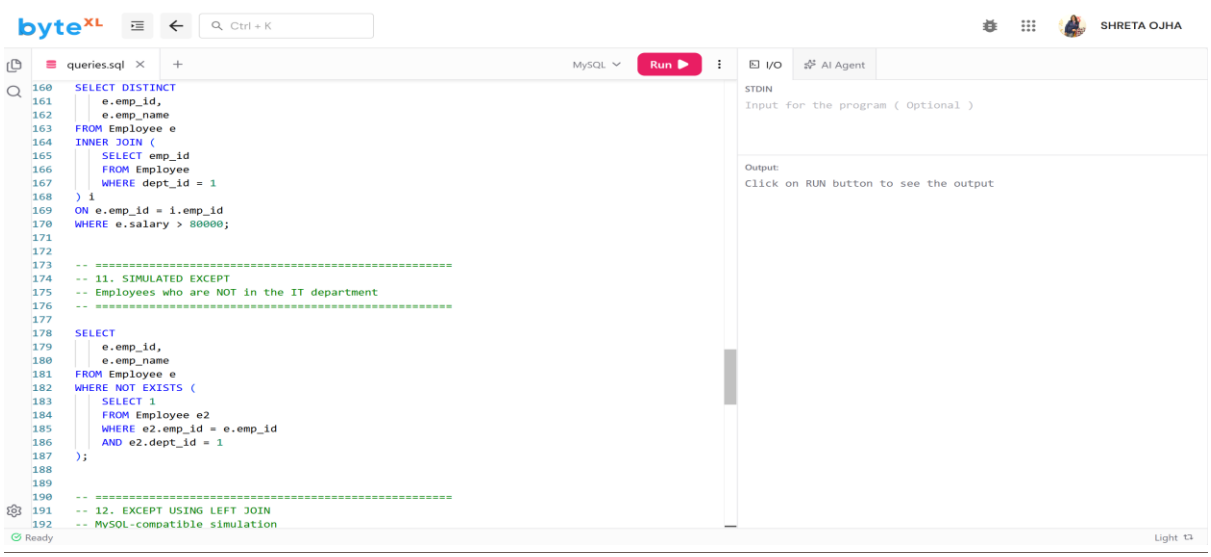

-- ======================================================
-- 9. SIMULATED INTERSECT
-- Employees who are in IT department
-- AND earn more than 80,000
-- ======================================================

SELECT emp_id, emp_name
FROM Employee
WHERE dept_id = 1

INTERSECT

SELECT emp_id, emp_name
FROM Employee
WHERE salary > 80000;


-- ======================================================
-- 10. INTERSECT USING INNER JOIN
-- MySQL-compatible simulation
```

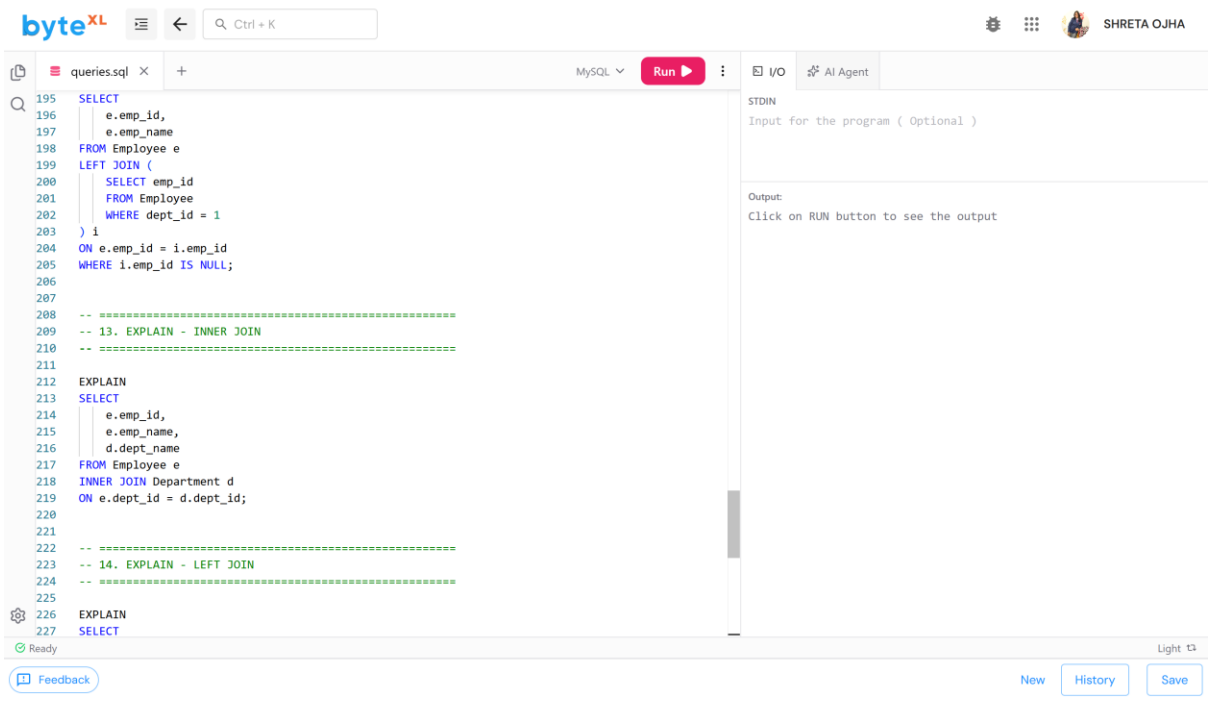



```sql
SELECT DISTINCT
    e.emp_id,
    e.emp_name
FROM Employee e
INNER JOIN (
    SELECT emp_id
    FROM Employee
    WHERE dept_id = 1
) i
ON e.emp_id = i.emp_id
WHERE e.salary > 80000;


-- ======================================================
-- 11. SIMULATED EXCEPT
-- Employees who are NOT in the IT department
-- ======================================================

SELECT
    e.emp_id,
    e.emp_name
FROM Employee e
WHERE NOT EXISTS (
    SELECT 1
    FROM Employee e2
    WHERE e2.emp_id = e.emp_id
    AND e2.dept_id = 1
);


-- ======================================================
-- 12. EXCEPT USING LEFT JOIN
-- MySQL-compatible simulation
```



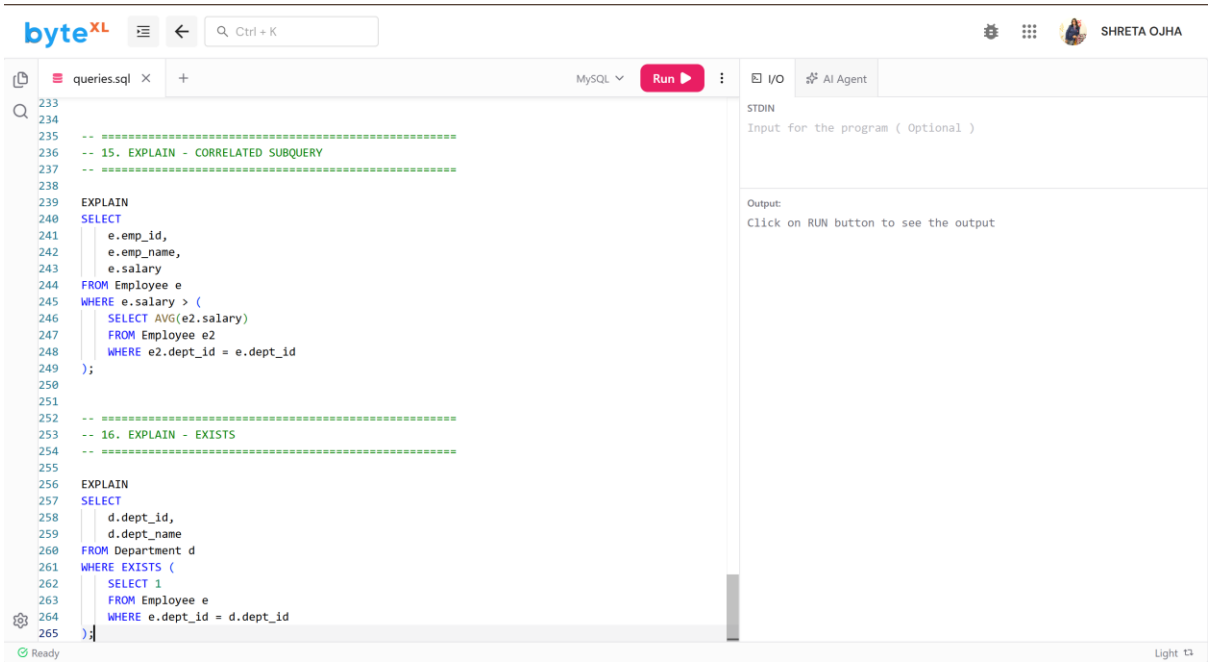

```sql
SELECT
    e.emp_id,
    e.emp_name
FROM Employee e
LEFT JOIN (
    SELECT emp_id
    FROM Employee
    WHERE dept_id = 1
) i
ON e.emp_id = i.emp_id
WHERE i.emp_id IS NULL;


-- ======================================================
-- 13. EXPLAIN - INNER JOIN
-- ======================================================

EXPLAIN
SELECT
    e.emp_id,
    e.emp_name,
    d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id;


-- ======================================================
-- 14. EXPLAIN - LEFT JOIN
-- ======================================================

EXPLAIN
SELECT
```



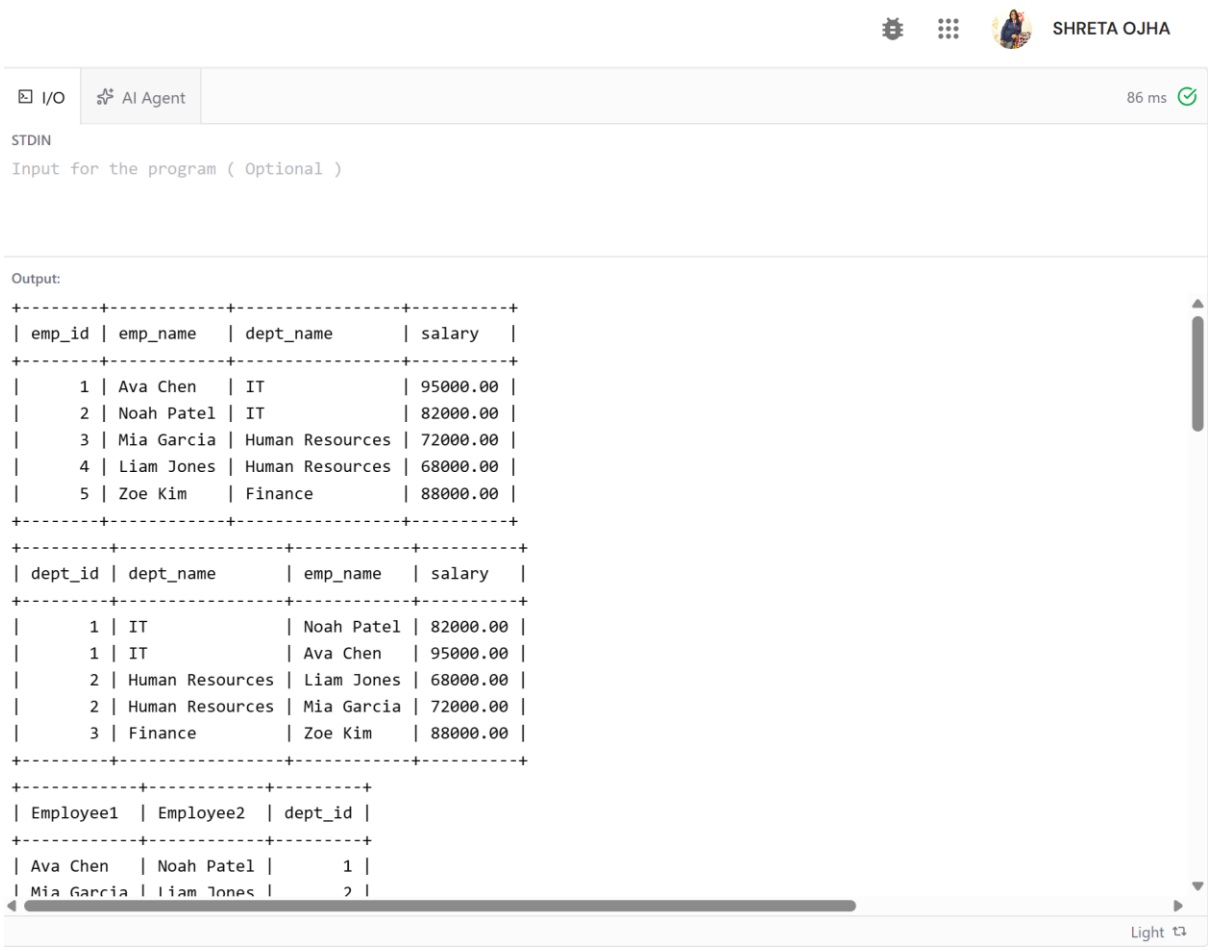
```sql
-- ================================================
-- 15. EXPLAIN - CORRELATED SUBQUERY
-- ================================================

EXPLAIN
SELECT
    e.emp_id,
    e.emp_name,
    e.salary
FROM Employee e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.dept_id = e.dept_id
);


-- ================================================
-- 16. EXPLAIN - EXISTS
-- ================================================

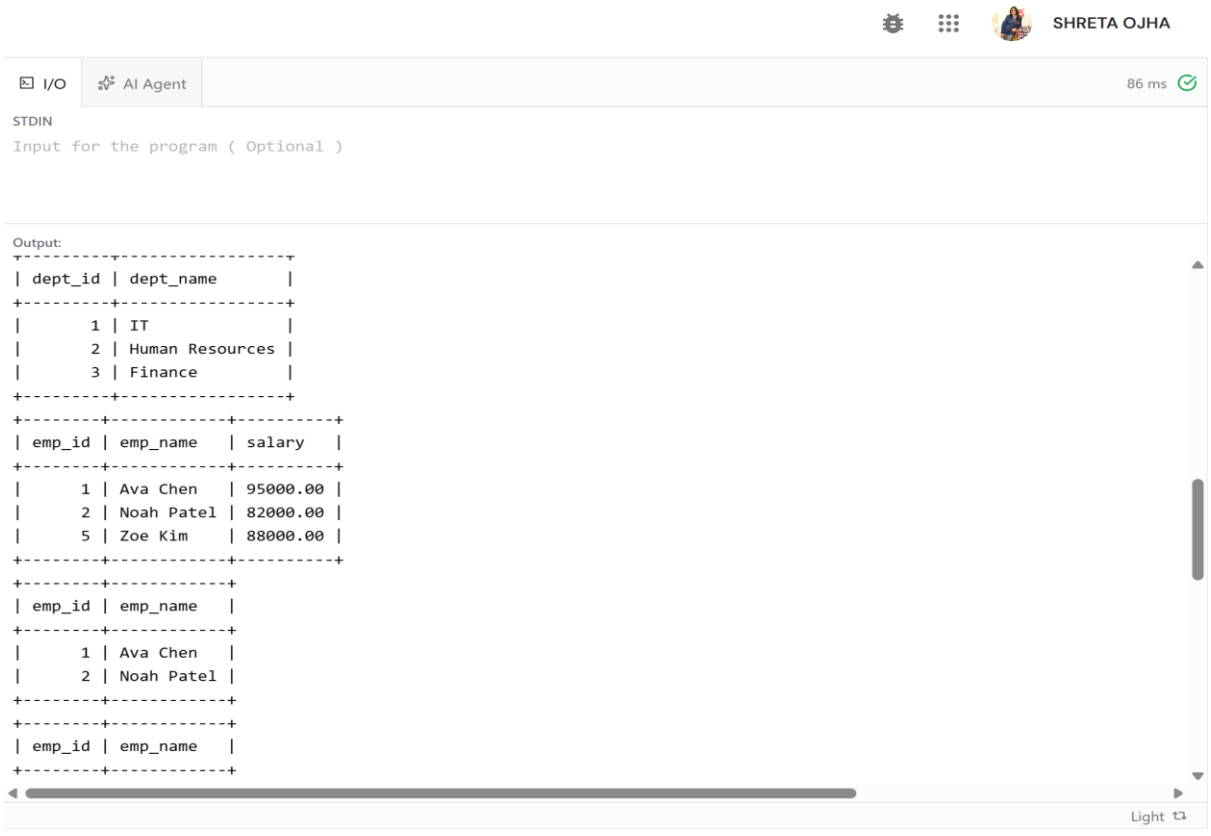
EXPLAIN
SELECT
    d.dept_id,
    d.dept_name
FROM Department d
WHERE EXISTS (
    SELECT 1
    FROM Employee e
    WHERE e.dept_id = d.dept_id
);
```

OUTPUT



```
+--------+------------+-----------------+----------+
| emp_id | emp_name   | dept_name       | salary   |
+--------+------------+-----------------+----------+
|      1 | Ava Chen   | IT              | 95000.00 |
|      2 | Noah Patel | IT              | 82000.00 |
|      3 | Mia Garcia | Human Resources | 72000.00 |
|      4 | Liam Jones | Human Resources | 68000.00 |
|      5 | Zoe Kim    | Finance         | 88000.00 |
+--------+------------+-----------------+----------+
+---------+-----------------+------------+----------+
| dept_id | dept_name       | emp_name   | salary   |
+---------+-----------------+------------+----------+
|       1 | IT              | Noah Patel | 82000.00 |
|       1 | IT              | Ava Chen   | 95000.00 |
|       2 | Human Resources | Liam Jones | 68000.00 |
|       2 | Human Resources | Mia Garcia | 72000.00 |
|       3 | Finance         | Zoe Kim    | 88000.00 |
+---------+-----------------+------------+----------+
+------------+------------+---------+
| Employee1  | Employee2  | dept_id |
+------------+------------+---------+
| Ava Chen   | Noah Patel |       1 |
| Mia Garcia | Liam Jones |       2 |
```


SHRETA OJHA
I/O
AI Agent
86 ms
STDIN
Input for the program (Optional)
Output:
+--------+------------+-----------------+---------------------+----------+
| emp_id | emp_name | dept_name | project_name | salary |
+--------+------------+-----------------+---------------------+----------+
1	Ava Chen	IT	Platform Upgrade	95000.00
2	Noah Patel	IT	Analytics Dashboard	82000.00
3	Mia Garcia	Human Resources	Recruitment Portal	72000.00
5	Zoe Kim	Finance	Analytics Dashboard	88000.00
+--------+------------+-----------------+---------------------+----------+
+--------+------------+----------+---------+
| emp_id | emp_name | salary | dept_id |
+--------+------------+----------+---------+
| 1 | Ava Chen | 95000.00 | 1 |
| 3 | Mia Garcia | 72000.00 | 2 |
+--------+------------+----------+---------+
+--------+------------+----------+---------+
| emp_id | emp_name | salary | dept_id |
+--------+------------+----------+---------+
1	Ava Chen	95000.00	1
3	Mia Garcia	72000.00	2
5	Zoe Kim	88000.00	3
+--------+------------+----------+---------+
+---------+-----------------+
| dept_id | dept_name |
Light


SHRETA OJHA
I/O
AI Agent
86 ms
STDIN
Input for the program (Optional)
Output:
+---------+-----------------+
| dept_id | dept_name |
+---------+-----------------+
1	IT
2	Human Resources
3	Finance
+---------+-----------------+
+--------+------------+----------+
| emp_id | emp_name | salary |
+--------+------------+----------+
1	Ava Chen	95000.00
2	Noah Patel	82000.00
5	Zoe Kim	88000.00
+--------+------------+----------+
+--------+------------+
| emp_id | emp_name |
+--------+------------+
| 1 | Ava Chen |
| 2 | Noah Patel |
+--------+------------+
+--------+------------+
| emp_id | emp_name |
+--------+------------+
Light



OBSERVATION/OUTCOME

1. Employee, Department, and Project tables were successfully created and populated with the required data.
2. INNER JOIN successfully retrieved matching employee and department records.
3. LEFT JOIN successfully displayed all departments along with their corresponding employees.
4. A self-join successfully identified employees belonging to the same department.
5. A 3-way join successfully retrieved employee, department, and project details together.
6. The correlated subquery successfully identified employees whose salary was greater than the average salary of their respective department.
7. The EXISTS operator successfully checked for the existence of projects associated with employees' departments.
8. INTERSECT and EXCEPT operations were successfully simulated using EXISTS and NOT EXISTS.
9. EXPLAIN successfully displayed the execution plans for different SQL queries, allowing their execution strategies to be examined and compared.